



Instituto Tecnológico de Mazatlán



Semáforo Peatonal con Botón usando Arduino UNO R4 WiFi

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Barrón Hernández

Integrantes:

Ximena Armenta Dávila

Carlos Tadeo Ibarra Castañeda

Dulce Princesa Ulivarria González

1. Introducción

Los semáforos peatonales con solicitud por botón son un ejemplo clásico de sistema de control secuencial, ya que combinan un ciclo de temporización fijo con un evento externo asíncrono que debe atenderse en el momento adecuado sin interrumpir dicho ciclo. En la presente práctica se diseñó e implementó un semáforo vehicular y peatonal utilizando un Arduino UNO R4 WiFi, en el que un pulsador permite a un peatón solicitar el cruce en cualquier momento, quedando dicha solicitud pendiente hasta que el semáforo vehicular llegue a rojo.

El sistema controla cinco LEDs: dos verdes, dos rojos y uno amarillo, distribuidos entre el semáforo vehicular (verde, amarillo y rojo) y el semáforo peatonal (rojo y verde). El objetivo fue programar y armar dicho control, verificando que la solicitud del peatón solo se atienda cuando fue presionada durante el verde o el amarillo de los autos, y que no tenga ningún efecto si se presiona durante el rojo vehicular.

2. Objetivos

Objetivo general:

- Diseñar e implementar un semáforo vehicular y peatonal controlado por un Arduino UNO R4 WiFi, en el que un pulsador permita solicitar el cruce peatonal de forma segura durante el ciclo vehicular.

Objetivos específicos:

- Definir la duración de cada estado del semáforo vehicular (verde, amarillo y rojo) y su correspondencia con los LEDs correspondientes.
- Implementar la lectura del pulsador con antirrebote por software, evitando lecturas falsas por rebote mecánico.
- Restringir la solicitud de cruce peatonal a los estados de verde o amarillo del semáforo vehicular, de forma que se atienda automáticamente al llegar a rojo.
- Armar el circuito físico en protoboard y verificar su comportamiento en los distintos escenarios de uso del botón.

3. Herramientas y material utilizado

- Arduino UNO R4 WiFi
- 5 LEDs (2 verdes, 2 rojos y 1 amarillo)
- 5 resistencias de $220\ \Omega$
- Pulsador (pushbutton) para la solicitud peatonal
- Protoboard y cables de conexión (jumpers)
- Arduino IDE

4. Desarrollo / Metodología

Los cinco LEDs se conectaron al protoboard a través de sus respectivas resistencias limitadoras de $220\ \Omega$. Los tres LEDs del semáforo vehicular (verde, amarillo y rojo) se conectaron a los pines digitales 8, 9 y 10 del Arduino, respectivamente, mientras que los dos LEDs restantes (rojo y verde) forman el semáforo peatonal. El pulsador se conectó a otro pin digital configurado como entrada, encargado de detectar la solicitud de cruce.

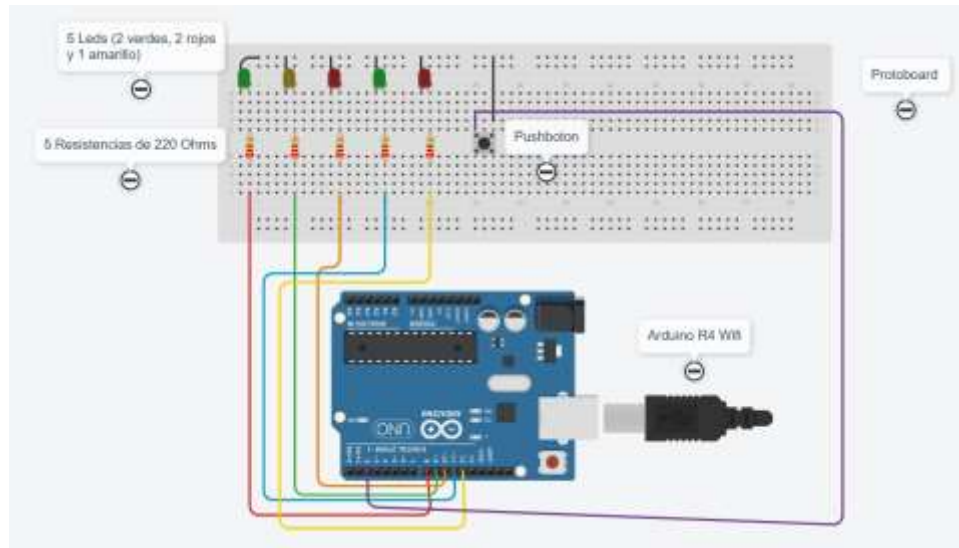


Figura 1. Diagrama esquemático del circuito: Arduino UNO R4 WiFi, protoboard, 5 LEDs, 5 resistencias de 220 Ω y pulsador.

El comportamiento del semáforo vehicular se programó como un ciclo fijo de 14000 ms, compuesto de 6000 ms en verde, 2000 ms en amarillo y 6000 ms en rojo, que se repite de forma continua sin depender del pulsador. La lectura del botón se implementó con un esquema de antirrebote por software de 40 ms, que evita que una sola pulsación física se interprete como varias solicitudes. La solicitud de cruce se guarda en una variable (solicitudPeaton) que solo se activa si el botón se presiona durante el verde o el amarillo de los autos; si se presiona durante el rojo vehicular, la pulsación no tiene efecto, ya que en ese momento la solicitud ya se resolvió o no existe una forma segura de atenderla. Una vez que el semáforo vehicular llega a rojo, la solicitud pendiente se atiende una sola vez y la variable se reinicia, de modo que una sola pulsación del botón solo genera un cruce peatonal.

5. Evidencia del circuito funcionando

A continuación, se presenta la fotografía del circuito armado en protoboard, con los cinco LEDs, las resistencias y el pulsador conectados, así como la conexión completa hacia el Arduino UNO R4 WiFi.

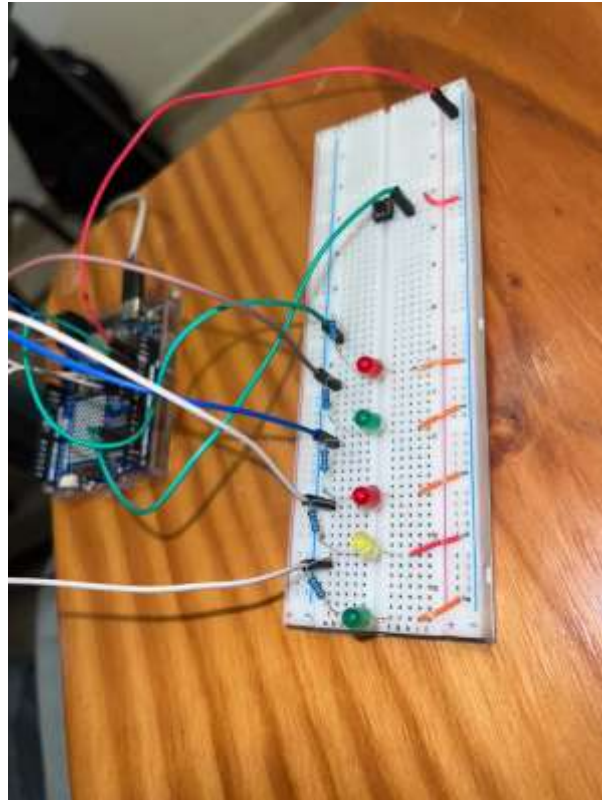


Figura 2. Circuito armado en protoboard: LEDs verdes, rojos y amarillo con sus resistencias, y pulsador para la solicitud peatonal.

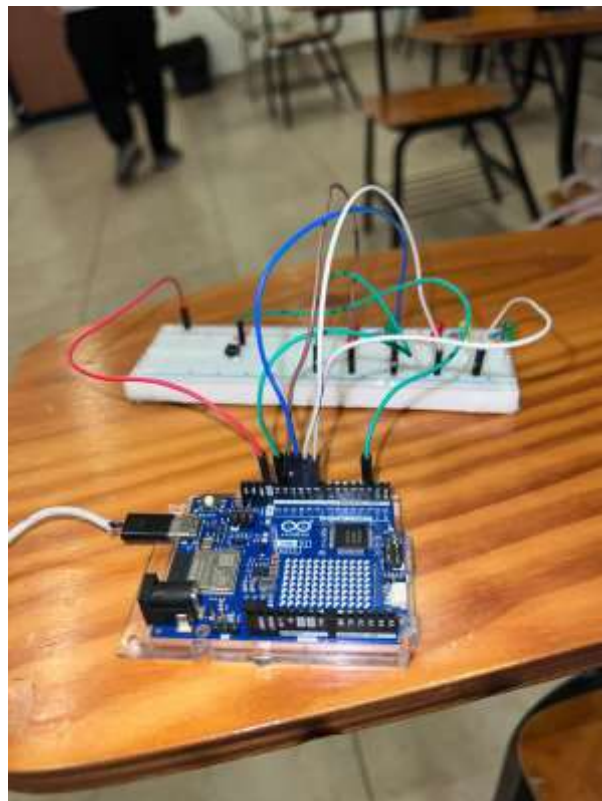


Figura 3. Conexión del protoboard al Arduino UNO R4 WiFi durante las pruebas de la práctica.

6. Análisis de resultados

Las pruebas se realizaron el 15 de septiembre de 2026. El comportamiento del semáforo se determinó a partir del código fuente (Codigo/Semaforo_Peatonal.ino) y se verificó en el armado físico, presionando el botón en distintos momentos del ciclo.

El semáforo de autos sigue siempre el mismo ciclo de 14000 ms (6000 ms en verde, 2000 ms en amarillo y 6000 ms en rojo). El semáforo peatonal depende de si se presionó el botón durante el verde o el amarillo de los autos. Las Figuras 4 y 5 muestran el estado de cada LED durante un ciclo completo, calculado directamente a partir de la lógica del programa, para los dos casos posibles.

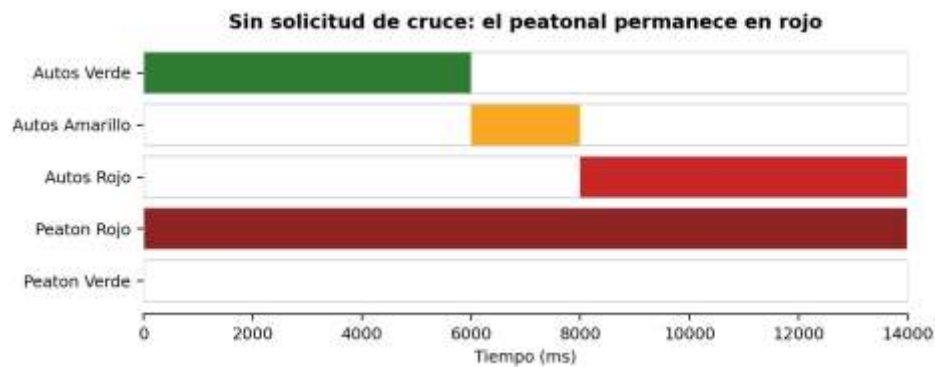


Figura 4. Sin solicitud de cruce: el semáforo peatonal permanece en rojo durante todo el ciclo.

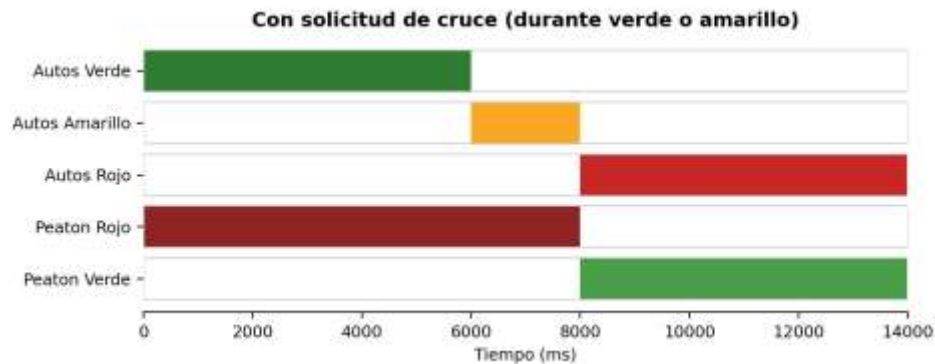


Figura 5. Con solicitud de cruce (botón presionado durante el verde o el amarillo de los autos): el semáforo peatonal se pone en verde exactamente cuando los autos llegan a rojo, y vuelve a rojo cuando los autos regresan a verde.

La Tabla I resume la duración de cada estado del semáforo de autos, definida en el código. La Tabla II compara el estado del semáforo peatonal en los dos escenarios mostrados en las Figuras 4 y 5.

Tabla I. Duración de cada estado del semáforo de autos.

Estado de autos	Duración	LED encendido
Verde	6000 ms	Verde de autos (pin 8)

Estado de autos	Duración	LED encendido
Amarillo	2000 ms	Amarillo de autos (pin 9)
Rojo	6000 ms	Rojo de autos (pin 10)

Tabla II. Estado del semáforo peatonal según el escenario.

Escenario	Semáforo peatonal
Sin solicitud de cruce	Rojo durante todo el ciclo (0–14000 ms)
Con solicitud durante verde o amarillo de autos	Rojo de 0 a 8000 ms; verde de 8000 a 14000 ms (mientras los autos están en rojo)
Solicitud presionada durante el rojo de autos	No tiene efecto: la solicitud solo se guarda si el botón se presiona en verde o amarillo

Observaciones sobre el comportamiento del sistema:

- El semáforo de autos nunca se detiene ni cambia su ciclo por el botón: los 14000 ms (6000 + 2000 + 6000) se repiten siempre, se presione o no el botón.
- El semáforo peatonal solo cambia de comportamiento si el botón se presionó durante el verde o el amarillo de los autos (estado 0 o 1); presionarlo durante el rojo de autos no tiene ningún efecto, porque en ese momento la solicitud ya se resolvió o no había forma de resolverla de manera segura.
- La solicitud de cruce se guarda en la variable solicitudPeaton y se atiende una sola vez, exactamente cuando el semáforo de autos pasa de amarillo a rojo; después la variable se reinicia a false, por lo que una sola pulsación del botón solo genera un cruce peatonal.
- El antirrebote (40 ms) evita que un solo toque del botón se interprete como varias pulsaciones, aunque no se midieron osciloscópicamente los rebotes reales del botón usado; el valor de 40 ms se tomó del código y es el que se usó en las pruebas.
- El armado físico y el video de la práctica confirman visualmente este comportamiento: el semáforo de autos corre solo, y el peatonal solo se pone en verde cuando se presionó el botón a tiempo.

7. Conclusiones individuales

Ximena

Esta práctica me permitió comprender cómo un sistema puede atender un evento externo, como la solicitud de un peatón, sin interrumpir ni alterar el ciclo principal que ya está en marcha. Ver que el semáforo vehicular sigue su temporización de 14000 ms sin importar si el botón se presiona o no me ayudó a entender la diferencia entre un evento que se atiende de inmediato y uno que se guarda para atenderse en el momento seguro.

Lo que más me llamó la atención fue la regla de que la solicitud solo se arma si el botón se presiona en verde o amarillo, y que se ignore por completo si se presiona en rojo. Al principio me pareció contraintuitivo que una pulsación pudiera "no hacer nada", pero analizando la lógica entendí que es precisamente esa restricción la que evita solicitudes redundantes o conflictivas, y que refleja cómo se diseñan reglas de seguridad en sistemas de control reales.

Carlos Tadeo

El desarrollo de esta práctica reforzó mi comprensión sobre el uso de variables de estado para representar un ciclo vehicular con distintas duraciones por color, y sobre la importancia de leer el pulsador en cada iteración del programa

sin bloquear la ejecución. Armar el circuito con cinco LEDs y sus resistencias, además del pulsador, me ayudó a practicar la organización del cableado en el protoboard para mantenerlo claro y fácil de revisar.

Me pareció relevante analizar las gráficas de las Figuras 4 y 5, ya que permiten comparar de forma visual el mismo ciclo de 14000 ms con y sin solicitud de cruce: en ambos casos el semáforo vehicular se comporta exactamente igual, y lo único que cambia es el estado del semáforo peatonal a partir del segundo 8000. Esto reforzó mi comprensión de que la solicitud del peatón no modifica el ciclo vehicular, solo determina qué LED peatonal se enciende durante el tramo en que los autos están en rojo.

Dulce Princesa

Considero que el mayor aprendizaje de esta práctica fue entender cómo el antirrebote por software resuelve un problema físico real del pulsador, evitando lecturas falsas que podrían generar solicitudes peatonales incorrectas. También quedó clara la importancia de definir con precisión en qué estados se permite "armar" una solicitud y en cuáles se debe ignorar, ya que esas reglas son las que garantizan que el sistema se comporte de forma segura y predecible.

Adicionalmente, comparar el escenario sin solicitud con el escenario con solicitud me ayudó a visualizar cómo una misma variable booleana (`solicitudPeaton`) puede cambiar por completo el comportamiento del semáforo peatonal sin afectar en nada al semáforo vehicular. Verificar este comportamiento tanto en el código como en el armado físico me permitió relacionar la teoría de las máquinas de estados con un resultado tangible y observable en los LEDs.

Referencias

[1] Arduino, "Arduino UNO R4 WiFi", documentación oficial. [En línea]. Disponible en: <https://docs.arduino.cc>

[2] Arduino, "Debounce", ejemplo oficial de antirrebote por software. [En línea]. Disponible en: <https://docs.arduino.cc/built-in-examples/digital/Debounce>